THM_LABS

SIMPLE CTF

End-to-End Penetration Testing: From Initial Access to Root Compromise

Prepared by

Ahmed Eldkrory (EDGAR)

Junior Penetration Tester

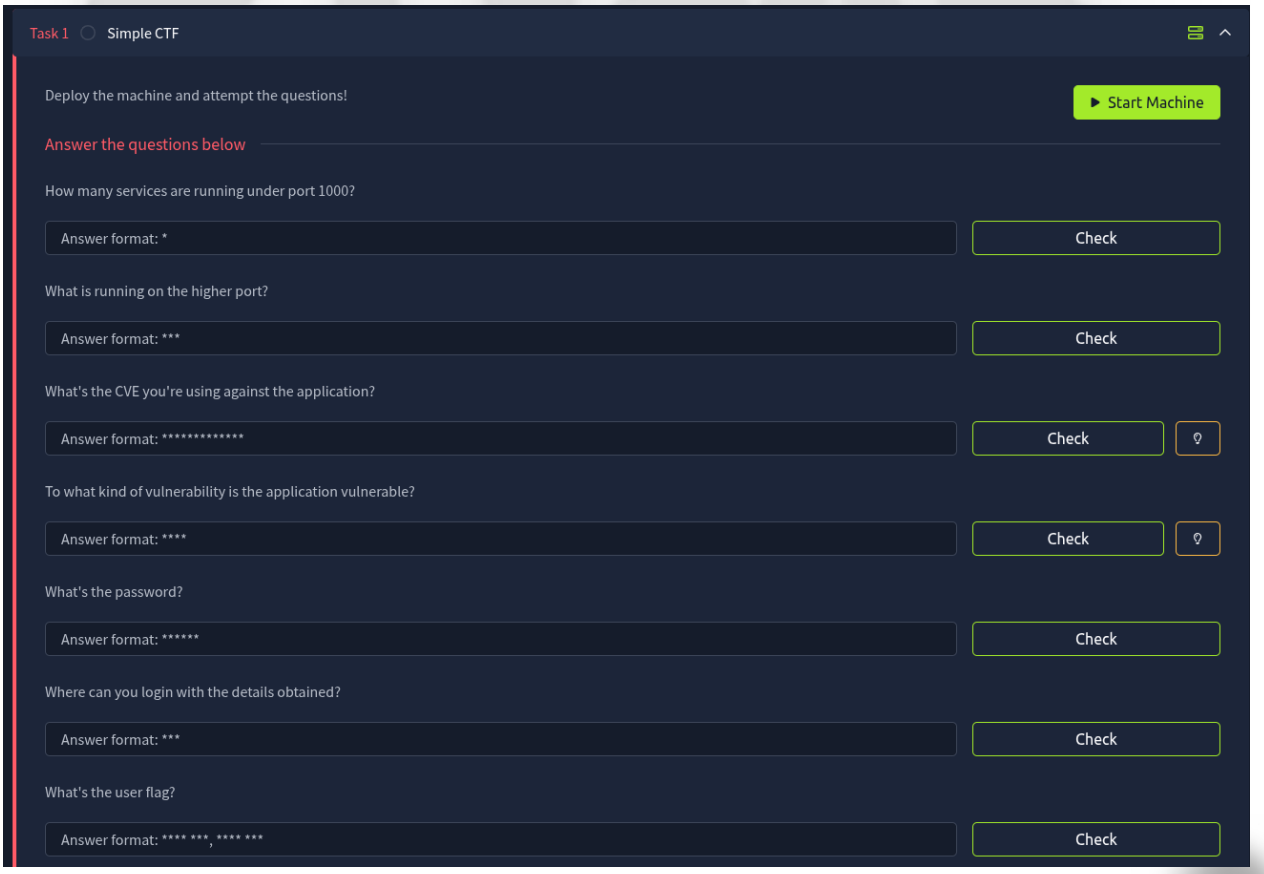
ahmed.a.eldkrory@gmail.com

Executive Summary

This assessment was conducted against a target system hosted at `10.114.180.121` as part of the **THM Labs – Simple CTF** environment. The room is designed to simulate a real-world penetration testing scenario, focusing on web application exploitation, credential compromise, and privilege escalation techniques.

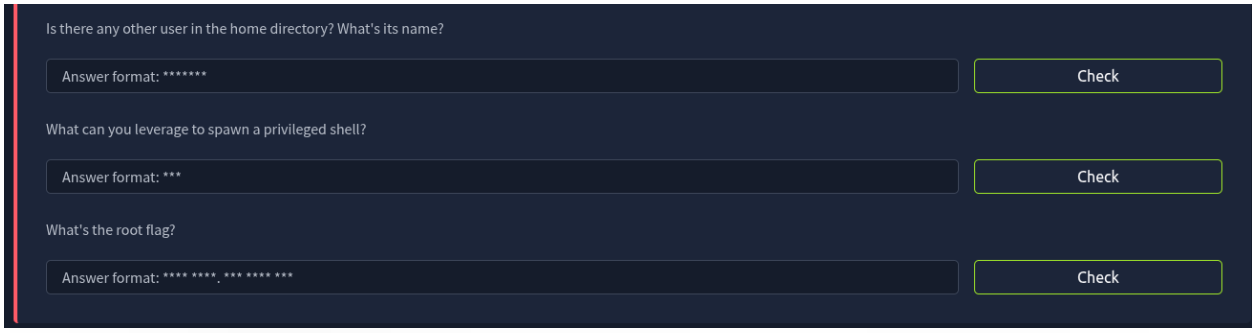
Full write-up: https://medium.com/@ahmed.a.eldkrory/thm-labs-simple-ctf-writeup-39355900e50a

The objective of this engagement was to identify security vulnerabilities, demonstrate realistic exploitation paths, and evaluate the potential impact of a successful compromise.



The assessment successfully resulted in full system compromise, beginning with unauthenticated access via a vulnerable web application and culminating in root-level privileges. The primary attack vector involved exploiting a known SQL Injection vulnerability in an outdated CMS platform. This allowed extraction of sensitive credentials, which were then leveraged to gain SSH access. Further privilege escalation was achieved through misconfigured sudo permissions.

Key findings indicate critical security weaknesses, including outdated software components, improper input validation, weak credential security, and excessive privilege assignment. These vulnerabilities, when chained together, enabled complete system takeover.



Scope & Target Information

Room Link: https://tryhackme.com/room/easyctf

Parameter	Value
Target IP	10.114.180.121
Environment	TryHackMe – Simple CTF
Assessment Type	Black-box Testing
Difficulty	Beginner

Methodology

The assessment followed a structured penetration testing methodology:

1. Reconnaissance

Initial information gathering to identify exposed services and attack surface.

2. Enumeration

Detailed probing of identified services to discover potential entry points and vulnerabilities.

3. Exploitation

Leveraging identified vulnerabilities to gain initial system access.

4. Privilege Escalation

Escalating access from a low-privileged user to full administrative (root) control.

Detailed Findings

1. Reconnaissance

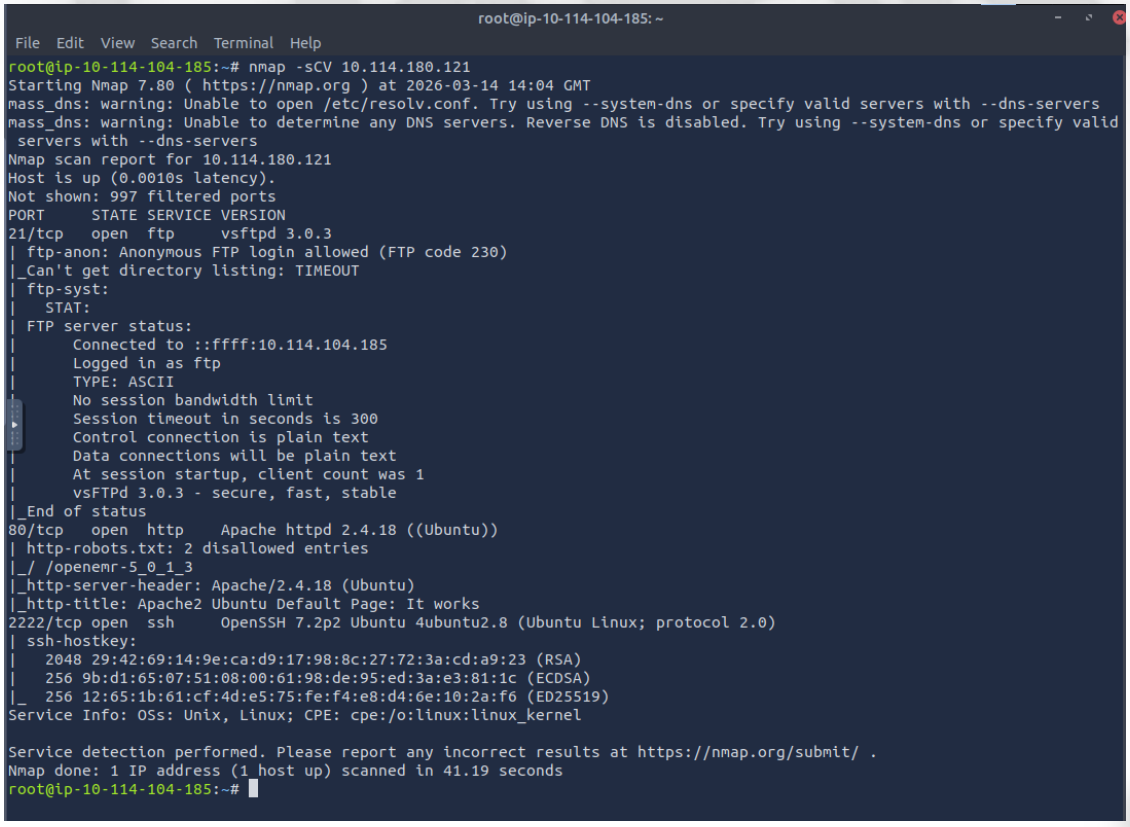
Network Scanning

A comprehensive scan was conducted using Nmap to identify open ports, running services, and potential vulnerabilities.

```
nmap -sCV 10.114.180.121
```

Purpose:

- `-sC`: Executes default scripts for basic vulnerability detection
- `-sV`: Performs service version detection



Findings:

Port	Service	Version	Observation
21	FTP	vsftpd 3.0.3	Anonymous login enabled
80	HTTP	Apache 2.4.18	Web server (Ubuntu)
2222	SSH	OpenSSH 7.2p2	Running on non-standard port

Analysis:
The presence of multiple exposed services expanded the attack surface. The HTTP service became the primary focus due to its high likelihood of hosting exploitable web applications.

---

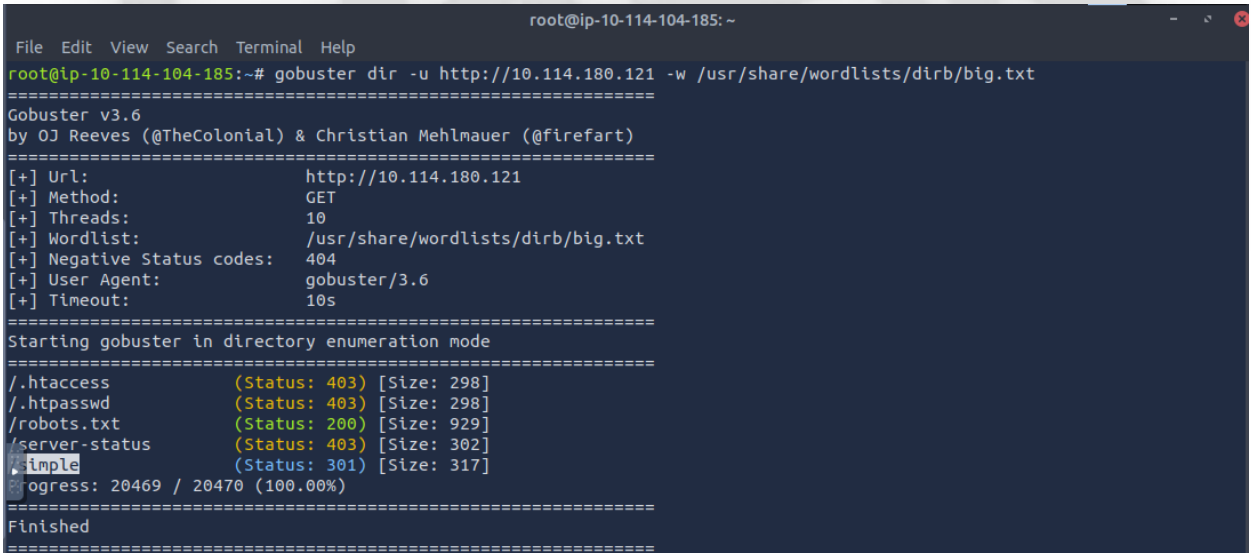
2. Enumeration

Web Directory Enumeration

Directory brute-forcing was performed to discover hidden resources:

```
gobuster dir -u http://10.114.180.121 -w /usr/share/wordlists/dirb/big.txt
```

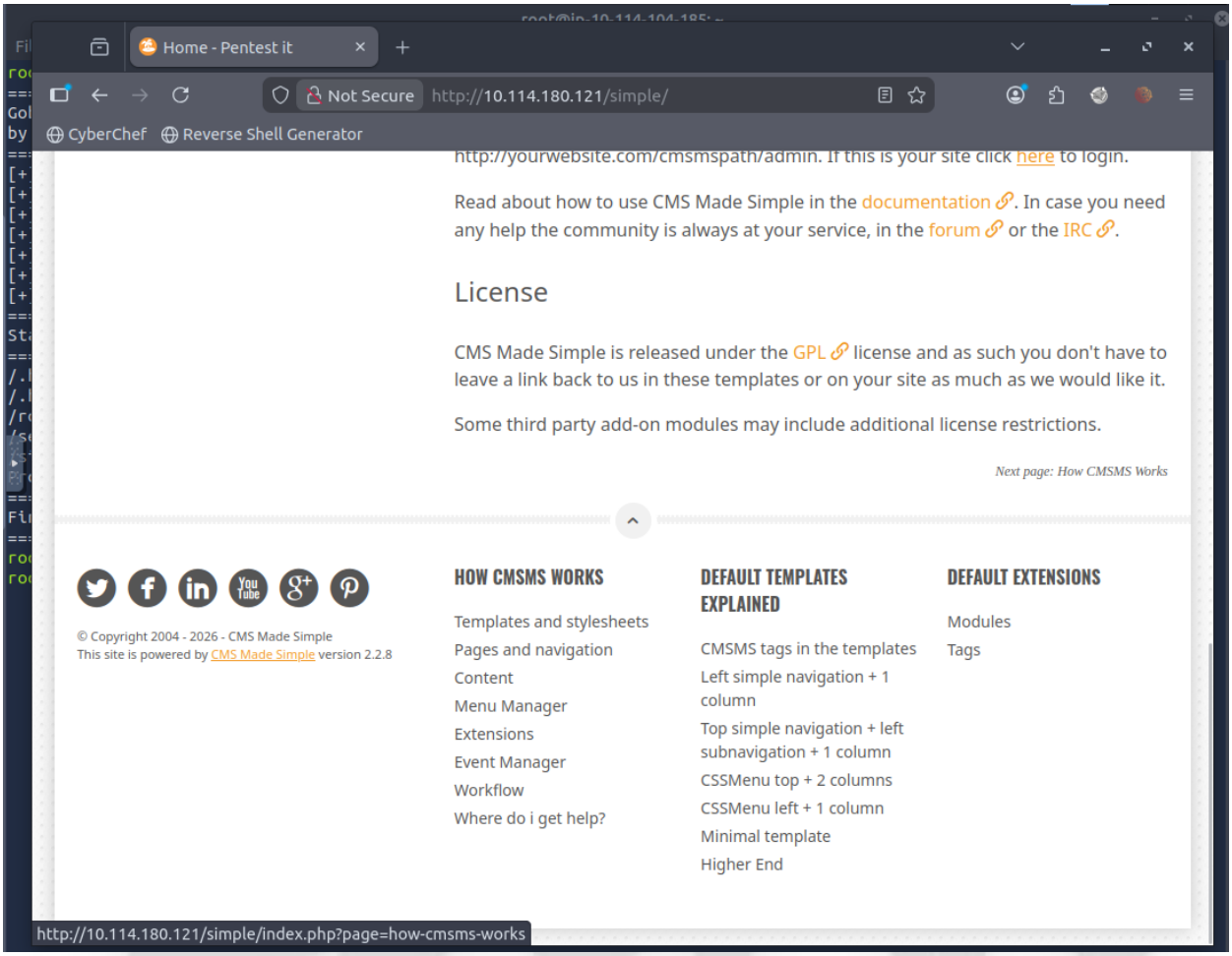
Purpose:
Gobuster is used to enumerate hidden directories and files that are not directly linked but may expose sensitive functionality.



Result:

- Discovered endpoint: `/simple`

Upon accessing this directory, it was identified as running **CMS Made Simple v2.2.8**.



Analysis:
Identifying the CMS version is critical, as outdated versions often contain publicly disclosed vulnerabilities. This discovery directly led to the next phase of exploitation.

---

3. Exploitation

Vulnerability Identification

The identified CMS version is vulnerable to CVE-2019-9053, an unauthenticated SQL Injection vulnerability.

Vulnerability Overview:

- Type: SQL Injection (Unauthenticated)
- Impact: Database extraction, credential disclosure
- Root Cause: Improper input sanitization in CMS parameters

Exploitation Execution

A publicly available Python exploit was used to extract sensitive database information:

Extracted Data:

- Username: `mitch`
- Salt: `1dac0d92e9fa6bb2`
- Password Hash: `0c01f4468bd75d7a84c7eb73846e8d96`

The hash was then cracked using a wordlist attack:

Cracked Credentials:

- Username: `mitch`
- Password: `secret`

Why the Attack Worked:

- The application failed to sanitize user input, allowing direct SQL query manipulation
- Sensitive data (password hashes) was accessible without authentication
- Weak password enabled successful offline cracking

---

4. Initial Access

Using the recovered credentials, SSH access was established:

```
ssh mitch@10.114.180.121 -p 2222
```

Analysis:

- SSH was running on a non-standard port (2222), which may evade basic scans but not comprehensive enumeration
- Credential reuse enabled direct system access without additional exploitation

---

5. Post-Exploitation & User Access

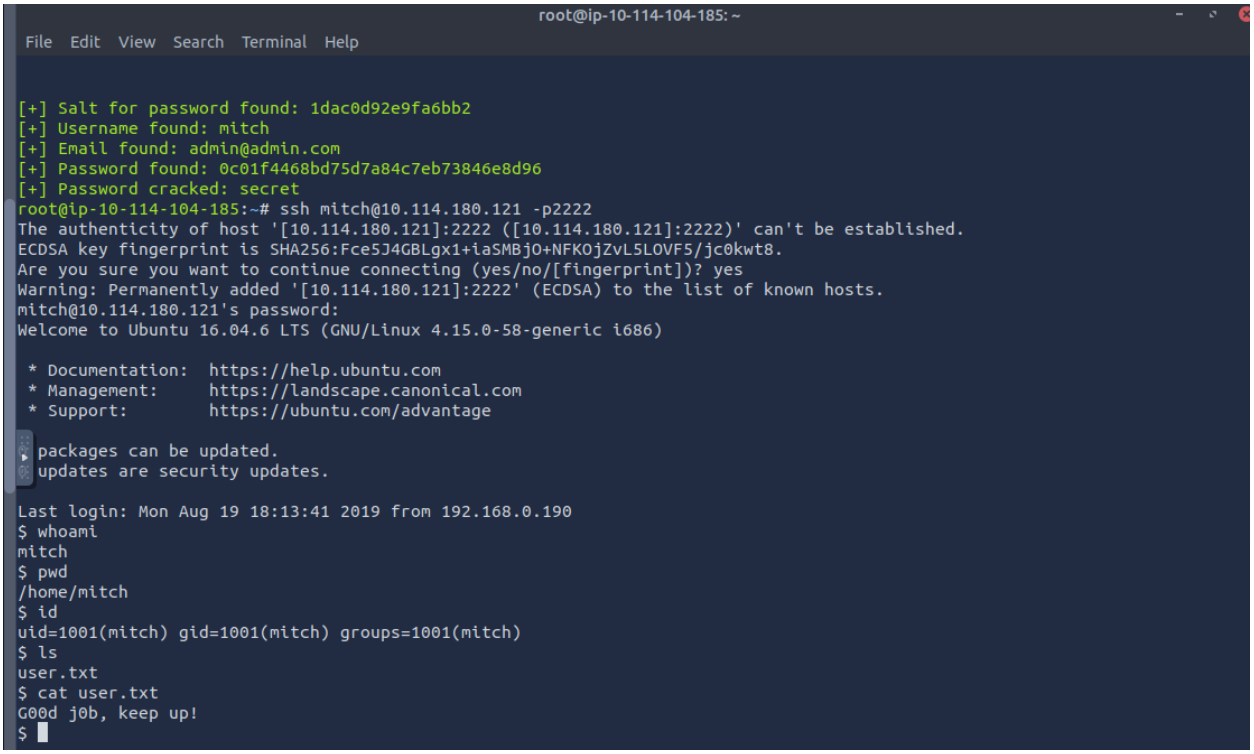
After successful login:

```
cat /home/mitch/user.txt
```

Observation:

- User flag successfully retrieved
- Additional user identified: `salt`

Analysis:
Presence of multiple users suggests potential lateral movement opportunities, though not required in this case.



---

6. Privilege Escalation

Sudo Privilege Enumeration

```
sudo -l
```

Finding:
The user `mitch` was allowed to execute `vim` as root without a password.

```
$ pwd
/home/mitch
$ cd ..
$ ls
mitch  sunbath
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$
```

Exploitation via GTFOBins

```
sudo vim -c ':!sh'
```

Purpose:
This command abuses Vim's shell execution capability to spawn a root shell.

Why the Attack Worked:

- Misconfigured sudo permissions allowed execution of a powerful binary
- Vim supports shell escape features, enabling command execution as root

Result:

- Full root shell obtained

```
$ sudo vim -c :!/bin/bash

                                VIM - Vi IMproved

                                 version 7.4.1689
                            by Bram Moolenaar et al.
                  Modified by pkg-vim-maintainers@lists.alioth.debian.org
                        Vim is open source and freely distributable

                               Help poor children in Uganda!
                    type  :help iccf               for information

                    type  :q                       to exit
                    type  :help         or         for on-line help
                    type  :help version7           for version inforoot@Machine:/home#
```

7. Root Access & Flag Retrieval

```
cat /root/root.txt
```

Outcome:

- Root flag successfully retrieved
- Complete system compromise achieved

```
root@Machine:/home# whoami
root
root@Machine:/home# cd ..
root@Machine:/# ls
bin   cdrom  etc   initrd.img      lib         media  opt   root  sbin  srv  tmp  var      vmlinuz.old
boot  dev    home  initrd.img.old  lost+found  mnt    proc  run   snap  sys  usr  vmlinuz
root@Machine:/# cd root
root@Machine:/root# ls
root.txt
root@Machine:/root# cat root.txt
W3ll d0n3. You made it!
root@Machine:/root#
```

Risk & Impact Analysis

1. SQL Injection (CVE-2019-9053)

Severity: Critical

Impact:

- Unauthorized database access
- Credential disclosure
- Full application compromise

---

2. Weak Credential Security

Severity: High

Impact:

- Enables brute-force or dictionary attacks
- Facilitates lateral movement and system access

---

3. SSH Access with Reused Credentials

Severity: High

Impact:

- Direct system access
- Bypasses additional authentication layers

4. Sudo Misconfiguration (Vim Execution)

Severity: Critical

Impact:

- Immediate privilege escalation to root
- Full system takeover

Remediation Recommendations

1. Patch Management

- Update CMS Made Simple to a secure version
- Regularly apply security patches

2. Input Validation & Secure Coding

- Implement parameterized queries
- Enforce strict input sanitization

3. Credential Security

- Enforce strong password policies
- Store passwords using strong hashing algorithms (e.g., bcrypt)

4. Access Control Hardening

- Restrict sudo permissions using the principle of least privilege
- Avoid allowing interactive binaries (e.g., Vim) with elevated privileges

5. Service Hardening

- Disable anonymous FTP access if not required
- Restrict SSH access (e.g., key-based authentication, IP whitelisting)

Conclusion

The assessment demonstrated how a chain of relatively common vulnerabilities—outdated software, weak credentials, and misconfigured privileges—can be combined to achieve full system compromise.

The initial foothold was gained through a known SQL Injection vulnerability, which exposed sensitive credentials. These credentials enabled SSH access, and a misconfigured sudo policy allowed immediate escalation to root privileges.

This highlights the importance of a defense-in-depth strategy, where multiple layers of security controls are implemented to prevent a single point of failure from leading to total compromise.
